FAILURE FORENSICS TOOL FOR AI PIPELINES

A Tracing + Root-Cause Analysis Layer for Multi-Step AI Systems

Project 3 of the AI Engineering Portfolio | Build Guide & Interview Companion

Author: Anas Amiar

"Treat the pipeline like a stack trace, not a black box. Record what every step received and produced. When the output is bad, find the first step that degraded — that's the root cause."

How to Use This Guide

This guide documents the Failure Forensics Tool exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview.

Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

An observability layer for a 4-step AI pipeline (intake, entity extraction, document classification, summarization). Every pipeline run produces a full Trace: one Span per step recording what went in, what came out, a confidence score, and a pass/fail status. When a pipeline produces bad output, the trace analyzer walks the spans in order, finds the first failed step (not just the most visibly broken one), and returns a named root cause with an evidence chain. Confirmed failures become permanent eval cases.

Running 8 documents (4 clean, 4 deliberately broken) through the pipeline: the analyzer correctly identified extraction as the root cause for the mixed-currency invoice, while correctly attributing the missing-date contract failure to propagation (the problem only became visible in summarization but originated in the data, not the extraction step).

WHY THIS PROJECT LANDS INTERVIEWS

Most pipeline demos show the happy path. This one shows what happens when something breaks, and — more importantly — shows a system that can tell you exactly where it broke and why. That's the observability gap most AI teams are genuinely running into right now: they have pipelines, and they have bad output sometimes, and they have no idea which step is at fault. Building a scoped-down LangSmith demonstrates that you understand the full lifecycle of an AI system in production, not just how to wire together API calls.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	Typed contracts: Span, Trace, RootCauseDiagnosis, etc.
Pipeline steps (mock)	Regex + keyword heuristics	Real, predictable failure modes — no API key needed
Persistence	JSON + SQLite	Human-readable trace files + queryable index
Root cause analysis	Forward trace walk	First failed span = root cause; downstream = symptom
Reporting	Static HTML	One page per trace, color-coded, root cause highlighted in r
Eval dataset	JSON (append-only)	Every diagnosed failure becomes a permanent regression te

The Four Pipeline Steps

Step	What it does	Designed failure mode
1. Intake	Accept raw document text	Near-empty document
2. Extraction	Pull out names, dates, amounts, key terms	Mixed currency symbols, no dates
3. Classification	Decide document type (contract/invoice/etc)	Ambiguous category wording
4. Summarization	Write a type-tailored one-line summary	Inherits upstream gaps (propagation)

Step-by-Step Build Guide

Six phases, in the order they were actually built. The pipeline had to come first and be strictly typed before tracing was layered in — spaghetti steps produce unattributable errors, which makes a forensics tool pointless. The test data had to have real failures built in before the analyzer had anything genuine to demonstrate.

PHASE 1: The 4-Step Pipeline + Sample Documents *Day 1*

Goal: Build clean, strictly-typed pipeline steps before adding any tracing overhead.

- Define typed input/output contracts for every step using Pydantic: `RawDocument`, `ExtractedEntities`, `ClassificationResult`, `SummaryResult`.
- Implement steps in `pipeline/steps.py` using regex (entity extraction) and keyword scoring (classification) — deterministic, no LLM calls needed.
- Build 8 sample documents in `data/documents.py`: 4 clean, 4 deliberately designed to break exactly one step (no dates, mixed currencies, ambiguous category, near-empty content).
- Run `python3 -m pipeline.run` to verify every step works on its own before wiring them together.

Deliverable: A working 4-step pipeline with typed contracts + 8 documents that produce organic, real failures.

PHASE 2: The Tracing Layer *Day 1-2*

Goal: Record exactly what every step received, produced, and scored — without touching the steps themselves.

- Define `Span` (step_name, input_summary, output_summary, confidence, status: success/failed).
- Define `Trace` (trace_id, doc_id, spans, final_status: success/failure/degraded).
- Orchestrator (`pipeline/orchestrator.py`): runs all 4 steps, builds one Span per step, sets final_status based on how many steps failed.
- Persist every Trace as JSON to `reports/traces/` and index it in SQLite for queryability.
- `LOW_CONFIDENCE_THRESHOLD = 2.5` — below this, a step is flagged even without a hard failure.

Deliverable: `pipeline/orchestrator.py` producing a full Trace per document, saved to JSON + SQLite.

PHASE 3: The Root-Cause Analyzer *Day 2*

Goal: Find where the problem actually started, not just where it was most visible.

- Walk spans IN ORDER and return the FIRST failed span — not the lowest-confidence one.
- A later step that fails because it inherited bad input should not be blamed for the problem.
- Map each step name to a failure category: extraction → "Extraction Failure", classification → "Misclassification", summarization → "Propagation Error".
- Return a `RootCauseDiagnosis` with the failed step, category, and an evidence chain string.

Deliverable: `pipeline/analyzer.py` — `diagnose(trace)` returning a named root cause with evidence.

PHASE 4: The Visual Trace Explorer *Day 2-3*

Goal: Make traces inspectable in a browser with zero setup.

- Generate one HTML page per document trace: each step shown as a colored block (green = success, yellow = degraded, red = failure).
- Root-cause step is highlighted and labeled explicitly.
- An index page lists all processed documents and links to their trace pages.
- All HTML is static and self-contained — `open reports/html/index.html` is all it takes.

Deliverable: `pipeline/explorer.py` + `reports/html/` — a clickable, browser-based trace explorer.

PHASE 5: The Feedback-to-Eval Loop *Day 3*

Goal: Turn diagnosed failures into a growing regression test suite.

- When a failure is flagged, `flag_failure(trace, original_text)` appends a structured entry to `reports/eval_dataset.json`.
- Each entry records: doc_id, step that failed, failure category, the original document text.
- `failure_analytics()` reads the full dataset and returns: total failures, breakdowns by step and category, most common failure point.
- This dataset grows over time and can be replayed against a new pipeline version to check whether known failures have been fixed.

Deliverable: `pipeline/feedback.py` — a persistent eval dataset + analytics showing which step fails most often.

PHASE 6: End-to-End Demo *Day 3-4*

Goal: Demonstrate the full forensics lifecycle on all 8 documents.

- Run all 8 documents through the orchestrator, generating 8 traces.
- Run the analyzer on all traces — verify it finds the right root cause for each broken document.
- Build the HTML explorer and verify the visual highlights match the analyzer's conclusions.
- Run the feedback loop and verify the analytics correctly identify extraction as the most common failure step.

Deliverable: 8 traces, 8 diagnoses, the HTML explorer, and an eval dataset that accurately reflects where the pipeline's weaknesses are concentrated.

A Real Bug Found During Development

While building the demo data, the regex-based name extractor flagged "This Agreement" as a person's name — the pattern just matches any two consecutive capitalized words, and the sentence started with "This Agreement is entered into between...". That's a textbook extraction hallucination, and it was caught by running the pipeline against real input. The fix was to reword the document text to avoid the false trigger — but the lesson is the same one the forensics tool itself teaches: heuristics need to be validated against real input, not just logic-checked in isolation.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Deciding that the root cause is the FIRST failed step, not the lowest-confidence one. My first instinct was to report whichever span had the worst confidence score — but that often pointed at a downstream step that was simply inheriting bad input from upstream, sending an engineer to debug the wrong code. Walking forward and stopping at the first failure finds the true origin, not just the loudest symptom. This is exactly how stack traces work in traditional debugging, and the insight was that the same principle applies to AI pipelines: blame the first thing that broke, not the last thing that cried.

What I'd Build Next

- Swap the mock regex steps for real LLM calls, and add LLM-as-judge confidence scoring instead of rule-based confidence.
- A live, clickable React trace explorer instead of static HTML.
- Automatic regression re-runs: periodically re-process the accumulated eval dataset against the current pipeline and track whether known failures have been fixed.

Companion Projects in This Portfolio

This project fills in the observability gap between the **Model Regression Detection System** (Project 1, which catches quality regressions at the prompt/model level) and the **LLM Cost Autopilot** (Project 2, which optimizes routing decisions). Together the three cover the core concerns of production AI: correctness, cost, and observability. The **OCR Document Intelligence Pipeline** (Project 4) extends the uncertainty-awareness theme to a different domain.

Repository: github.com/Anas-Amiar/Project-3-failure-forensics-tool